Guide complet — RHOAI 3.5 MaaS Demo (Sopra Steria)

Ce document unique couvre tout le cycle : sandbox neuve → installation → identifiants → démo live (clic par clic) → nettoyage → dépannage. Toute la doc "comment faire" du repo est ici, volontairement en un seul fichier pour ne pas avoir à deviner où chercher.

Durée démo live : ~20 minutes (Demo 1 : publication native, ~5 min ; Demo 2 : gouvernance/parcours utilisateur, ~15 min).

Sommaire

1. Prérequis
2. Sandbox neuve → tout installer
3. Identifiants — qui, où, comment
4. Créer les clés API (dashboard)
5. Checklist avant d'entrer en salle
6. Démo live — parcours complet, clic par clic
7. Aide-mémoire (post-it)
8. Ce qu'il ne faut PAS dire ou promettre
9. Dépannage
10. Nettoyage
11. Annexe technique

---

1. Prérequis

Élément	Détail
Sandbox OpenShift	RHOAI 3.5.x déjà installé (ou installable), accès `cluster-admin`
Outils locaux	`oc` , `git` , `bash` , `make` , `curl` , `python3` , `dig`
Réseau	Accès pull vers `registry.redhat.io` et `ghcr.io`

Tu n'as **jamais** besoin de coller ton token OpenShift dans Git, dans un fichier du dépôt, ou dans un chat.

---

2. Sandbox neuve → tout installer

En une commande (recommandé)

```
oc login --server=<API> --token=<TOKEN>
git clone https://github.com/Lindagh1/rhoai-maas-demo.git && cd rhoai-maas-demo
bash scripts/bootstrap-redeployment.sh
```

Ça installe, de façon idempotente (relançable sans casser quoi que ce soit) : - **Demo 1** : opérateurs requis, activation MaaS, observabilité (Cluster Observability Operator + OpenTelemetry + RBAC), modèle natif (`LLMInferenceService`), publication (`MaaSModelRef`) - **Demo 2** : identité de test non-admin (`maas-demo2-user`), groupe (`maas-demo2-developers`), gouvernance (`MaaSSubscription` + `MaaSAuthPolicy`)

Puis affiche les étapes manuelles restantes (section 4 de ce guide).

Options : `--demo1-only` , `--demo2-only` , `--skip-observability` .

Pas-à-pas, si tu veux du contrôle (équivalent sans le script unique)

```
make demo1-all              # opérateurs, MaaS, observabilité, modèle, publication, véri
make demo2-detect-idp       # détecte le fournisseur d'identité du cluster (lecture seul
make demo2-create-identity  # crée maas-demo2-user + groupe maas-demo2-developers
make demo2-grant-access     # applique MaaSSubscription + MaaSAuthPolicy
make demo2-audit            # vérifie l'état avant de continuer
```

Validation, sans rien muter

```
bash scripts/validate-redeployment.sh
```

Vérifie que chaque manifest actif passe `oc apply --dry-run=server` contre les CRDs réellement installées sur ce cluster — aucune écriture.

---

3. Identifiants — qui, où, comment

Admin

C'est le compte avec lequel tu as fait `oc login` / que tu utilises pour te connecter à la console OpenShift et au dashboard RHOAI en tant qu'administrateur. Rien à récupérer : c'est le tien.

`maas-demo2-user` (utilisateur final non-admin)

- **Login :** `maas-demo2-user`
- **Mot de passe :** généré **une seule fois** par `make demo2-create-identity` (ou par `bootstrap-redeployment.sh`, qui l'appelle), stocké en local uniquement, **jamais commité** (`secrets/` est dans `.gitignore`), dans : `text secrets/demo2/maas-demo2-user.password` Pour le lire : `bash cat secrets/demo2/maas-demo2-user.password`
- **Si le fichier est vide ou absent** alors que l'utilisateur existe déjà sur cette sandbox (ex. créé par un run précédent dont le fichier local a été perdu) :
- **Keycloak in-cluster** (le cas le plus courant sur ces sandboxes) : relance simplement `make demo2-create-identity` — le script détecte l'absence du fichier local et réinitialise le mot de passe via l'API admin de Keycloak.
- **HTPasswd** : le mot de passe ne peut **pas** être retrouvé depuis le hash stocké. Il faut soit demander à qui l'a créé, soit supprimer la ligne `maas-demo2-user:<hash>` du secret htpasswd (`openshift-config`) et relancer `make demo2-create-identity` .
- **OIDC externe (Entra ID, etc.)** : voir §11.5 — procédure manuelle, ce script refuse de modifier un IdP d'entreprise partagé.

Clés API (créées par `maas-demo2-user` , voir section 4)

Une fois créées et capturées, elles vivent en local dans :

```
secrets/demo2/maas-demo2-user.apikey
secrets/demo2/maas-demo2-user.ratelimit.apikey
```

---

4. Créer les clés API (dashboard)

C'est la **seule** étape volontairement non-automatisée du parcours Demo 2 — c'est justement le geste qu'on démontre (révélation one-shot d'une clé dans le dashboard), donc on ne la scripte pas.

Pas à pas, dans le dashboard

1. Ouvre le dashboard RHOAI : `https://rhods-dashboard-redhat-ods-applications.<domaine-cluster>`
2. Connecte-toi en tant que **`maas-demo2-user`** (mot de passe : §3) — idéalement dans une fenêtre de navigation privée, pour garder ta session admin ouverte dans une autre fenêtre.
3. Menu de gauche → **Gen AI studio** → **API keys**
4. Onglet **API keys** → bouton **Create API key**
5. Dans le formulaire : - **Name** : par exemple `maas_llm_demo` - **Subscription** : sélectionner `maas-demo2-subscription` - **Expiration** : laisser la valeur par défaut (max 90 jours sur cette version) ou choisir une date

6. Valider → la **valeur de la clé n'est affichée qu'une seule fois**. Copie-la immédiatement.
7. **Répéter l'opération une deuxième fois**, en sélectionnant cette fois la subscription `maas-demo2-subscription-ratelimit-test` (clé dédiée pour la démonstration du rate-limit, étape 6.6 plus bas).

Capturer les clés en local (sans jamais les coller en clair ailleurs)

```
make demo2-capture-keys
```

Ce target lance `scripts/demo2/21-capture-api-keys.sh`, qui demande les deux valeurs via une saisie masquée (`read -s`) et les écrit directement dans `secrets/demo2/maas-demo2-user.apikey` et `secrets/demo2/maas-demo2-user.ratelimit.apikey` (chmod `600`, gitignorés).

Vérifier que tout est vert

```
make demo2-verify
```

Doit afficher `[OK] All checks passed` avec les deux clés en `200` (actives). Si tu vois `HTTP status: 000` ou un échec, voir §9 (Dépannage).

---

5. Checklist avant d'entrer en salle

- [ ] `make demo1-verify` → `11 passed`
- [ ] `make demo2-verify` → `All checks passed`
- [ ] Les 2 clés API sont actives (pas expirées)
- [ ] 3 onglets navigateur prêts : | Onglet | URL | Compte | |---|---|---| | Console OpenShift | `https://console-openshift-console.<domaine-cluster>` | admin | | Dashboard RHOAI (admin) | `https://rhods-dashboard-redhat-ods-applications.<domaine-cluster>` | admin | | Dashboard RHOAI (user) | même URL, fenêtre privée | `maas-demo2-user` |
- [ ] Un workbench utilisateur prêt (ou au moins le projet `rhoai-maas-demo` visible côté `maas-demo2-user`)
- [ ] Un terminal admin ouvert dans le repo (pour restaurer un état si besoin pendant la démo)

Si quelque chose n'est pas vert, c'est le moment de le corriger — pas pendant la démo.

---

6. Démo live — parcours complet, clic par clic

Fil rouge à dire au début

> « Demo 1 montre comment **publier** un modèle en MaaS. Demo 2 montre comment **gouverner son usage** : qui a accès, avec quelle limite, ce qui se passe quand ça casse, et ce que voient vraiment l'admin et l'utilisateur final. »

```
Console OpenShift (YAML)          Dashboard OpenShift AI (clics)
        │                                  │
        ├─ LLMInferenceService ──────── Gen AI studio → AI asset endpoints (modèle visib
        ├─ MaaSModelRef ─────────────── Gen AI studio → AI asset endpoints → badge MaaS
        ├─ MaaSSubscription ─────────── Settings → MaaS governance → Subscriptions (admi
        │                               Gen AI studio → API keys → Subscriptions (user)
        ├─ MaaSAuthPolicy ──────────    Settings → MaaS governance → Authorization polic
        │                               Gen AI studio → API keys → Create API key (user)
        └─ Gateway ──────────────────── curl /v1/chat/completions → 200
```

Étape 0 (optionnelle) — Sans accès, l'utilisateur ne peut rien faire

À montrer seulement si tu pars volontairement d'un état "sans gouvernance" (`make demo2-cleanup-access CONFIRM=yes` au préalable).

Session `maas-demo2-user` → **Gen AI studio** → **API keys** : - Onglet Subscriptions : *"Request a subscription — ... contact your administrator."* - Onglet API keys : *"No API keys"* - **Create API key** quand même : alerte *"No subscriptions available"*, et le sélecteur de subscription a `pointer-events: none` (réellement désactivé, pas juste grisé visuellement).

> « Sans subscription admin, un vrai utilisateur non-admin ne peut même pas créer une clé exploitable. »

Étape 1 — Le modèle natif (Demo 1, ~5 min)

Console OpenShift : 1. Menu gauche → **Search** → Resources : `LLMInferenceService` 2. Namespace : `rhoai-maas-demo` 3. Clique sur `demo-simulator` → onglet **YAML** 4. Montre : `kind: LLMInferenceService` (chemin natif llm-d, pas legacy KServe), `spec.model.name` , `spec.router.gateway.refs` → `maas-default-gateway` 5. Onglet **Details** → `Ready: True`

> « C'est le manifeste qui déploie le modèle — du YAML standard, versionnable en Git. »

↔ **Dashboard RHOAI (admin) :** 1. **Gen AI studio** → **AI asset endpoints** → onglet Models → projet `rhoai-maas-demo` 2. Montre `demo-simulator` avec ses endpoints (External + Internal)

> ⚠️ **Ne pas chercher dans Projects → Model Deployments** — les `LLMInferenceService` natifs n'y apparaissent pas, seulement les `InferenceService` legacy KServe. La bonne vue est **Gen AI studio**.

↔ **Montrer le geste admin en clics (formulaire, sans valider) :**

> ⚠️ Ne pas utiliser « Edit model deployment » sur l'existant (le formulaire sera vide, le modèle a été déployé par YAML). Utiliser « **Deploy model** » (nouveau).

1. Projet `rhoai-maas-demo` → bouton **Deploy model**
2. Page 1 — **Model location** : `URI` ; **URI** : `hf://sshleifer/tiny-gpt2` ; **Name** : `demo-simulator` → **Next**
3. Page 2 — cocher **Distributed inference with llm-d** (pas « Use legacy deployment method ») → **Next**
4. Page 3 — descendre à **Model availability**, montrer la case ☑ **Publish as MaaS** (équivalent clic du `MaaSModelRef`)
5. → **Cancel** ⛔ (ne jamais cliquer Deploy ici — le GPU est déjà pris et ça créerait un doublon)

Console OpenShift — `MaaSModelRef` : 1. Search → `MaaSModelRef` → namespace `rhoai-maas-demo` → `demo-simulator` 2. YAML : `spec.modelRef.kind: LLMInferenceService` 3. Details : `phase: Ready`, conditions `GovernancePaired`, `RuntimeHealthy`

> « C'est la pièce qui publie le modèle dans le catalogue MaaS — même résultat que la case "Publish as MaaS" vue à l'instant. »

Étape 2 — La gouvernance, vue admin (~3 min)

Console OpenShift — `MaaSSubscription` (le COMBIEN) : 1. Search → `MaaSSubscription` → namespace **`models-as-a-service`** (pas celui du modèle — c'est le namespace du tenant MaaS) 2. `maas-demo2-subscription` → YAML : `yaml spec: owner: groups: - name: maas-demo2-developers modelRefs: - name: demo-simulator tokenRateLimits: - limit: 2000 window: 1m`

Console OpenShift — `MaaSAuthPolicy` (le QUI) : 1. Search → `MaaSAuthPolicy` → namespace `models-as-a-service` → `maas-demo2-access` → YAML : `spec.subjects.groups` / `spec.modelRefs`

> « Subscription = combien. AuthPolicy = qui. Les deux sont nécessaires ensemble — sans l'une des deux, c'est un 403. »

↔ **Dashboard RHOAI (admin) :** 1. **Settings** → **MaaS governance** → onglet **Subscriptions** : montre `maas-demo2-subscription` `Active`, groupes, modèles, quota 2. Onglet **Authorization policies** : montre `maas-demo2-access` `Active`, groupes autorisés, modèles

> « L'admin peut créer/modifier/supprimer directement depuis cette interface, pas seulement en YAML. »

Si besoin de remettre l'état correct à tout moment :

```
make demo2-grant-access
```

Étape 3 — L'utilisateur crée sa clé (~2 min)

Session `maas-demo2-user` → **Gen AI studio** → **API keys** : 1. Onglet **Subscriptions** : la subscription est maintenant visible avec son quota (`2,000 / 1 minute`) 2. **Create API key** → le sélecteur de subscription est **actif** (preuve que l'AuthPolicy fonctionne) 3. Sélectionner `maas-demo2-subscription`, nommer la clé, valider 4. **La valeur n'est affichée qu'une seule fois** — c'est le point à verbaliser

> « La preuve côté utilisateur : il peut créer une clé. Sans l'AuthPolicy, ce sélecteur serait désactivé. »

Étape 4 — Inférence réelle depuis un vrai workbench utilisateur (~2 min)

La partie la plus pédagogique : l'utilisateur appelle le modèle depuis son propre environnement, pas depuis un terminal admin.

Dashboard utilisateur : 1. **Projects** → **`rhoai-maas-demo`** 2. Ouvrir un workbench existant (ou en créer un), ouvrir son terminal 3. Vérifier que la variable d'environnement contenant la clé est bien présente : `echo "${#DEMO2_API_KEY}"` (si `0`, voir §9)

Commandes dans le terminal du workbench :

```
curl -sk --http1.1 -H "Authorization: Bearer $DEMO2_API_KEY" \
  https://maas.apps.<domaine>/v1/models | python3 -m json.tool

cat > /tmp/body.json << 'JSON'
{"model": "publishers/rhoai-maas-demo/models/facebook/opt-125m", "messages": [{"role":"u
JSON

curl -sk --http1.1 -H "Authorization: Bearer $DEMO2_API_KEY" \
  -H "Content-Type: application/json" --data-binary @/tmp/body.json \
  https://maas.apps.<domaine>/v1/chat/completions -w "\nHTTP %{http_code}\n"
```

À montrer : `GET /v1/models` → `200`, `POST /v1/chat/completions` → `200` avec une vraie réponse et un vrai bloc `usage`.

⚠️ Toujours `--http1.1` depuis un pod/workbench. Jamais de JSON inline avec `-d "{\"model\"..."` dans un terminal web (échappement imprévisible) — toujours passer par un fichier (`--data-binary @fichier`).

Équivalent scripté admin (pour vérification rapide, pas pour le live) :

```
make demo2-test-inference
```

Étape 5 — Ce qui se passe quand la gouvernance casse (~3 min)

Pour chaque cas : **supprimer en click-ops**, **prouver par l'appel utilisateur**, **restaurer via manifest**.

5a. Supprimer l'Authorization policy

- Admin : **Settings** → **MaaS governance** → **Authorization policies** → supprimer `maas-demo2-access`
- Utilisateur : rejouer le `POST /v1/chat/completions` → **`HTTP 403`** (`GET /v1/models` peut rester en `200` avec une liste vide — la vraie preuve est l'appel d'inférence, pas `/v1/models`)
- Restauration : `oc apply -f manifests/demo2/11-maas-authpolicy.yaml`

5b. Supprimer la Subscription principale

- Admin : **Settings** → **MaaS governance** → **Subscriptions** → supprimer `maas-demo2-subscription`
- Utilisateur : rejouer l'appel → **`HTTP 403`** avec le corps `"requested subscription not found"` (pas `429` comme certaines docs pourraient le laisser penser)
- Restauration : `oc apply -f manifests/demo2/10-maas-subscription.yaml`

5c. Démontrer le rate-limit

Utiliser la **clé dédiée** à `maas-demo2-subscription-ratelimit-test` (créée à l'étape 4 du §4) :

```
cat > /tmp/body.json << 'JSON'
{"model": "publishers/rhoai-maas-demo/models/facebook/opt-125m", "messages": [{"role":"u
JSON

for i in $(seq 1 30); do
  curl -sk --http1.1 -H "Authorization: Bearer $RATELIMIT_KEY" \
    -H "Content-Type: application/json" --data-binary @/tmp/body.json \
    https://maas.apps.<domaine>/v1/chat/completions \
    -o /dev/null -w "call $i -> HTTP %{http_code}\n"
done
```

À montrer : quelques `200`, puis des `429`.

> « Le quota n'est pas décoratif : la plateforme finit réellement par refuser les appels. »

Équivalent scripté admin (3 tests négatifs, restaure automatiquement) :

```
make demo2-test-negative
```

Étape 6 — Qui voit quoi comme usage (~2 min, être honnête ici)

Côté utilisateur (Observe & Monitor → Dashboard, ou API keys / Subscriptions) : voit bien le modèle, l'onglet **Models**, et la limite configurée sur sa subscription. Ne voit **pas** sa consommation restante en temps réel ni une vue par clé.

Côté admin (Observe & Monitor → Dashboard → Models, éventuellement Usage) :

> « L'onglet **Usage** existe visuellement sur cette build, mais peut rester à **0** même après du vrai trafic — les métriques Limitador portent des labels de subscription (et parfois de modèle), mais pas de label utilisateur exploitable par les requêtes de cet onglet. L'onglet **Models**, en revanche, fonctionne bien et vaut le détour. »

Preuve technique hors pitch (compteurs Prometheus réels de Kuadrant/Limitador) :

```
make demo2-check-usage
```

Étape 7 — Révoquer et régénérer une clé (~1 min)

Session `maas-demo2-user` : 1. Sur une clé active → menu ⋮ → **Revoke** → confirmer 2. Rejouer `GET /v1/models` ou `POST /v1/chat/completions` → **`HTTP 403`** (pas le `401` que certaines docs annoncent — constat réel sur cette build) 3. Créer une nouvelle clé → la remettre dans l'environnement du workbench → rejouer l'appel → repart immédiatement en `200`

Clôture (30 s)

> « Récap : chaque pièce est un CR Kubernetes versionnable en Git. L'interface OpenShift AI montre le résultat utilisateur, la console OpenShift montre le YAML technique. Toute cette démo est rejouable en une commande : `bash scripts/bootstrap-redeployment.sh`. »

7. Aide-mémoire (post-it)

```
INSTALLATION
  bash scripts/bootstrap-redeployment.sh
  cat secrets/demo2/maas-demo2-user.password   (login maas-demo2-user)
  Dashboard (user) -> Gen AI studio -> API keys -> Create API key (x2)
  make demo2-capture-keys && make demo2-verify

DEMO LIVE
  Console: Search -> LLMInferenceService -> demo-simulator (YAML + Ready)
  Dashboard: Gen AI studio -> AI asset endpoints -> demo-simulator
  Dashboard: Deploy model (NOUVEAU, pas Edit) -> URI + llm-d + Publish as MaaS -> Cancel
  Console: Search -> MaaSModelRef -> demo-simulator (YAML + GovernancePaired)
  Console: Search -> MaaSSubscription / MaaSAuthPolicy -> ns models-as-a-service
  Dashboard admin: Settings -> MaaS governance -> Subscriptions / Authorization policies
  Dashboard user: Create API key -> sélecteur actif -> créer
  Workbench: curl /v1/models puis /v1/chat/completions -> 200
  Admin supprime AuthPolicy -> user: 403
  Admin supprime Subscription -> user: 403 "subscription not found"
  User (clé ratelimit) -> boucle 30x -> 200 puis 429
  Dashboard: Observe & Monitor -> Models (fiable) / Usage (peut rester à 0)
  Dashboard user: Revoke -> 403, nouvelle clé -> 200
```

8. Ce qu'il ne faut PAS dire ou promettre

❌ À éviter	Pourquoi
Dire "401" pour une clé révoquée	Le vrai code observé est **403**
Promettre un suivi d'usage **par utilisateur**	Limitador n'expose pas de label utilisateur exploitable sur cette build
Montrer "AI asset endpoints" pour créer une clé temporaire	Ça renvoie un vrai `403`, même pour le propriétaire du projet
Chercher `demo-simulator` dans Projects → Model Deployments	Les `LLMInferenceService` natifs n'y apparaissent pas — utiliser **Gen AI studio → AI asset endpoints**
Utiliser "Edit model deployment" sur `demo-simulator`	Formulaire vide (modèle déployé par YAML) — utiliser "Deploy model" (nouveau)
Cliquer "Deploy" dans le wizard au lieu de "Cancel"	Le GPU est déjà pris, ça échouerait ou créerait un doublon
Laisser tourner `demo2-test-negative` sans prévenir	~90s, le client peut croire que c'est planté
Improviser dans les formulaires dashboard pour restaurer vite	Les manifests du repo (`oc apply -f manifests/demo2/...`) sont plus sûrs

---

9. Dépannage

"Login failed" / mauvais cluster

`oc whoami --show-server` doit contenir l'identifiant **de ta** sandbox. Si ce n'est pas le cas : **arrête**, ne continue pas.

GPU déjà occupé

Normal sur les sandboxes avec un seul GPU déjà pris par un modèle legacy — la démo bascule automatiquement sur un simulateur CPU (`llm-d-inference-sim`), même chemin natif, sans toucher au modèle existant.

`MaaSModelRef` reste en `Pending`

Sur RHOAI 3.5.2, son statut global attend la paire `MaaSSubscription` / `MaaSAuthPolicy` (Demo 2). Pour valider seulement Demo 1 : vérifier `LLMInferenceService Ready` + condition `RuntimeReady=True`, c'est suffisant.

Mot de passe de `maas-demo2-user` vide ou introuvable

Voir §3 — relancer `make demo2-create-identity` (auto-reset sur Keycloak) ou régénérer manuellement sur HTPasswd.

`make demo2-verify` échoue avec `HTTP status: 000`

Cause réelle déjà rencontrée : le fichier de clé capturé contient du "bruit" de collage terminal autour de la vraie valeur `sk-oai-...`, produisant un header `Authorization` malformé (échec de transport, pas un vrai code HTTP). Les scripts extraient désormais la clé par regex (`read_api_key_file()` dans `scripts/demo2/lib.sh`) — si ça persiste, vérifie la taille du fichier (`wc -c secrets/demo2/*.apikey` — une vraie clé fait ~60-70 caractères) et recapture avec `make demo2-capture-keys` .

`echo "${#DEMO2_API_KEY}"` renvoie `0` dans le workbench

La variable d'environnement n'a pas été injectée. Depuis un terminal admin :

```
bash scripts/demo2/22-wire-workbench-api-key.sh <namespace> <notebook-name> secrets/demo
```

L'onglet Usage reste à 0 même après du vrai trafic

Limite connue de cette build, pas un bug de configuration — voir §6 et §11.4. Utiliser l'onglet **Models** ou `make demo2-check-usage` comme preuve fiable à la place.

---

10. Nettoyage

Retirer uniquement la gouvernance Demo 2 (garde le modèle et l'identité) :

```
make demo2-cleanup-access CONFIRM=yes
```

Retirer les ressources Demo 1 (modèle, publication — ne touche pas à RHOAI/RHCL, plateforme partagée) :

```
CONFIRM=yes bash scripts/99-cleanup.sh
```

---

11. Annexe technique

11.1 Automatisé vs volontairement manuel

Automatisé et idempotent (sans danger de relancer) : audit lecture seule, installation/activation des opérateurs et de MaaS, activation de l'observabilité, déploiement + publication du modèle natif, création de l'identité de test, application de la gouvernance, toutes les vérifications.

Volontairement manuel (ne sera jamais scripté, par conception) : - **Création des clés API** côté utilisateur final (§4) — c'est le cœur de la démo - **Création du Workbench** depuis le dashboard — le wiring interne (image, PVC, sidecar `kube-rbac-proxy`, ServiceAccount, HTTPRoute) géré par RHOAI n'a jamais été reproduit depuis du YAML brut ; seul le **patch** d'un Workbench déjà créé est automatisé (`scripts/demo2/22-wire-workbench-api-key.sh`) - **Révocation/régénération de clé** (§6.7) — c'est le geste UI qu'on démontre

11.2 Paramètres qui varient entre sandboxes

Aucun n'est codé en dur — tous détectés dynamiquement via `scripts/lib.sh` / `scripts/demo2/lib.sh` :

Paramètre	Comment il est déterminé
Domaine du cluster / hostname MaaS	`oc get ingresses.config.openshift.io cluster`
Certificat TLS par défaut	ingress controller, fallback `router-certs-default`
Namespace infra MaaS	`MaasTenantConfig.status.infraNamespace`, fallback `redhat-ai-gateway-infra`
Fournisseur d'identité (Demo 2)	lecture de `oauth/cluster` : htpasswd / Keycloak in-cluster / OIDC externe
GPU disponible	allocatable − requêtes réelles des pods ; bascule auto sur simulateur CPU si `0`

11.3 Chaînes fonctionnelles

```
Demo 1 : LLMInferenceService (llm-d) → MaaSModelRef → modèle publié dans MaaS
Demo 2 : MaaSSubscription (combien) + MaaSAuthPolicy (qui) → clé API utilisateur
         → inférence gouvernée (quotas, 403/429, révocation)
```

11.4 Limites connues de cette version (RHOAI 3.5.0/3.5.2)

- Une clé API révoquée renvoie `403`, pas le `401` documenté.
- Supprimer la subscription d'une clé renvoie `403` "subscription not found", pas `429`.

- L'onglet **Observe & Monitor → Dashboard → Usage** peut rester à `0` même après du vrai trafic : les métriques Limitador n'exposent pas de label `user` exploitable sur cette build. L'onglet **Models** est en revanche fiable.
- Le chemin "AI asset endpoints" pour une clé temporaire est bloqué par un vrai `403`, même pour un propriétaire de projet.
- L'étape "Environment variables" du wizard de création de Workbench ne s'applique pas toujours réellement — utiliser le script de patch dédié (§9).

11.5 Identité externe (OIDC d'entreprise, ex. Entra ID)

`scripts/demo2/01-create-test-identity.sh` détecte automatiquement le type de fournisseur d'identité (`htpasswd` / `keycloak-incluster` / `oidc-other`) via `oauth/cluster`, et **refuse délibérément** de créer un compte automatiquement sur un IdP externe/partagé (ex. Entra ID, Red Hat SSO d'entreprise) — modifier cette infrastructure n'est pas quelque chose qu'un script doit faire sans intervention humaine.

Procédure manuelle dans ce cas : 1. Créer un compte de test non-admin dans l'IdP externe (en dehors de ce repo, selon le processus de l'entreprise) 2. Vérifier que le mapping de claim de groupe est configuré avant de cibler une `MaaSSubscription` / `MaaSAuthPolicy` par `Group` OpenShift — sans ce mapping, le groupe OpenShift ne sera jamais peuplé automatiquement depuis les claims OIDC 3. Un aller-retour navigateur réel par `/oauth/token/request` reste la seule façon d'obtenir un token CLI pour cette identité (un provider OIDC/redirect ne supporte jamais `oc login -u/-p` directement)

11.6 Sécurité — rappels non négociables

- Aucun secret en clair n'est jamais committé (`secrets/` est dans `.gitignore`).
- Aucune clé API ni token OpenShift n'est jamais collé dans le chat, dans un log, ou dans un argument de commande visible à l'écran.
- Tout token/clé exposé par erreur (collé dans le chat, visible sur une capture d'écran) doit être considéré compromis et régénéré.
- Les Gateways partagés et les modèles legacy existants ne sont jamais écrasés par les scripts de ce repo.

11.7 Documentation officielle Red Hat (3.5)

- Govern LLM access with MaaS
- Distributed inference with llm-d
- Supported configurations

11.8 Résultats détaillés / preuves de test

Pour la matrice complète des scénarios testés (PASS/BLOCKED/FAIL, constats réels vs. documentation officielle) : voir `DEMO2-RESULTS.md`.